Programación asíncrona en JavaScript

La **programación asíncrona** se basa en que **ciertas tareas pueden ser ejecutadas en paralelo a otras** y, por tanto, que una instrucción siga a dichas tareas no implica que se ejecute después de la misma.

Promesas

Las promesas **permiten gestionar tareas asíncronas de una manera organizada**. Una promesa es un objeto que indica que cierta información va a ser proporcionada en un futuro. La promesa no especifica cuándo se va a retornar dicha información pero sí que se ejecutará una función en cuanto esté disponible u otra cuando no lo esté.

Una `Promesa` se encuentra en uno de los siguientes estados:

- *pendiente (pending)*: estado inicial, no cumplida o rechazada.
- *cumplida (fulfilled)*: significa que la operación se completó satisfactoriamente.
- *rechazada (rejected)*: significa que la operación falló.

Una promesa se crea con:

```
let promesa = new Promise(function(resolve,reject){
    …
    resolve(valor);
    …
    reject(valor);
    …
}
```

NOTA: NO es obligatorio que las funciones callback se llamen así, pero suele ser lo habitual. A la promesa se le pasa una función con dos parámetros que son a la vez funciones.

Una vez definida la promesa, podemos ejecutar sobre ella los métodos `then()`, `catch()` y `finally()`.

En la promesa primero se programa lo que se quiere hacer(generalmente serán operaciones costosas). Con resolve se le manda resolver de forma existosamente con un resultado y con reject erroneamente con un error)

El método `then(fncOk,fncErr)` se ejecuta cuando finaliza la promesa. Se le pueden pasar dos callbacks (uno para cuando la promesa ha tenido éxito y otro para cuando no)

```
let p = new Promise(function(resolve,reject){
    let tot = 0;
    for(let i=0;i<100;i++){
        tot+=i;
    }
    resolve(tot);
});

p.then(
    result => console.log(result),
    error => console.log("Error : " + error)
);

console.log("Sigue el código");
```

La salida

```
Sigue el código
4950
// inspeccionado p se visualiza
p
>Promise {<fulfilled>: 4950} >
```

Si provocamos un error: Estaria entrando por el then en la pate de error, porque solo hemos especificado la función reject. Se resuleve la promesa de manera erronea

```
p = new Promise((resolv,reject) => {
    reject("Datos erróneos");
})
p.then(
    result => console.log(result),
    error => console.log("Error : " + error)
);
console.log("Sigue el código");
```

La salida

```
Sigue el código
Error : Datos erróneos
// inspeccionado p se visualiza
p
 > Promise {<rejected>: 'Datos erróneos'}
```

NOTA: Como se puede ver en el ejemplo, aunque el código de la promesa es inmediato (ya que no hace nada muy costoso), se ejecuta después del resto de código (del hilo principal).

NOTA: La función de error también captura los errores producidos dentro de la promesa sean del tipo que sean.

La cláusula `catch()` permite capturar los errores producidos en la promesa:

```
p = new Promise((resolv,reject) => {
    reject("Datos erróneos");
});
// En el siguiente ejemplo el error en la promesa lo gestiona con el catch ( ya que en el then solo le pasamos la funcion de ok (result))
p.then(
    result => console.log(result)
)
.catch(
    err => console.log(err)
);
```

La salida

```
Sigue el código
Datos erróneos
```

En realidad, un `catch` hace exactamente lo mismo que la función de `onRejected.(Segundo parámetro que se pasa en el then)`

Es por esto que si existe la función `reject` y el código `catch` y se produce un `reject` el código `catch` no se ejecuta, tiene prioridad el `onRejected`

Por último, la cláusula `finally()` se ejecuta siempre se haya resuelto o rechazado la promesa y se hayan producido errores o no.

```
// ejemplo, si p fuera una promesa
p
  .then(result => console.log(result))
  .catch(err => console.log(err))
  .finally(() => console.log("Por aqui siempre paso, independientemente"))

// Normalmente el finally en otros lenguajes de programacion es muy usado para
// cerrar recursos (ficheros, conexiones a base de datos, etc)
// para asegurar que independientemente de que haya ido bien o mal, se libera el recurso
```

Async / Await

Cuando trabajamos con promesas podemos emplear `async / await` para simplificar su uso.

- `async` : Pemite etiquetar una función como asíncrona, lo que quiere decir que retorna una promesa. El valor de retorno de dicha función se empaqueta como una promesa. Es una forma elegante de solucionar el problema de cuando queremos pasar parámetros a una promesa

Esta función podemos reescribirla con `async` de la siguiente forma:

Al ser una funcion async, automaticamente retorna una promesa que se resuelve con el return

```
async function sumar(num) {

  let total = 0;
  for (let i = 1; i <= num; i++) {
     total += i;
  }

  return total;
}
```

El resultado, evidentemente es el mismo

- `await` : La palabra reservada `await` permite **esperar a la finalización de una promesa**. Si lo usamos dentro de una funcion, esta tiene que ser async, y desde el año 2022 también puede ejecutarse en el hilo principal.

```
console.log(await sumar(100));

console.log(await sumar(10000));

console.log("Resultados");

//La salida vemos que resultado está abajo porque el await espera a que se finalice la función y retorne la promesa. Si hubieramos puesto el .then() no
```

```
esperaría y en este caso Resultados se vería primero. Realmente el await espera a que acabe la función y retorne algo(es decir, como si lo convirtiera en sincrono)

// Con el async que hace que una función sea asincrona se puede utilizar en await o el then.

// se utiliza más el async/await , aunque hay veces que debes utilizar el then


> 500500

> 50005000

> Resultados
```

Recuperando información de internet con Fetch

Una de las tareas asíncronas más habituales es **recuperar información de internet**. Para ello, existen **varios mecanismos** pero, probablemente, el **más simple y versátil** sea la función `fetch`.

En su forma más básica, `fetch` **permite recuperar datos de un servidor y retorna una promesa con dicha información.**

Generalmente para obtener información de Internet se realiza a través de una URL(endpoint de una API(es decir una URL para hacer algo sobre la API)).

Normalmente el formato de respuesta es JSON o XML. El JSON es el más utilizado el día de hoy, ocupa menos espacio y es más legible.

Tambien existen API's a las cuales se les puede pasar el formato que se quiere de respuesta.

Por ejemplo, vamos a usar un API simple (https://dog.ceo/dog-api/documentation/) para ver el funcionamiento, que nos devuelve un JSON.
En esta documentación del API, podemos ver los dos diferentes endpoints que nos ofrece para realizar diferentes tareas. Para obtener una **lista de razas de perros** emplearíamos la URL https://dog.ceo/api/breeds/list/all. Vemos como podemos hacerlo con `fetch`:

```
// con esta línea recuperamos la información (nos devuelve un objeto Promesa cuyo resultado será un objeto Response y lo almacenamos en  la variable f)
```

NOTA: para probar estas peticiones , se deben hacer desde el liveserver. Esta API sólo se puede ejecutar desde un servidor. Por eso abrimos el live server

```
let f = fetch("https://dog.ceo/api/breeds/list/all");

// definimos dos funciones callback (una para el caso de éxito, cuando todo va bien, y otra para el caso de error)

// result contendrá ya el objeto resultante de la promesa, un objeto Response. El objeto Response contendrá además de la información, codigo http de estado de la respuesta(status), cabeceras, formato de la respuesta, etc

f.then(
```

```
    result => console.log(result),
    error => console.log("Error : " + error)
);
```

La salida:



NOTA: Para ver esta información vamos a la consola, a Network y los códigos de estado (status entre 200 y 300 es correcto. Por ejemplo, el 200 es OK) se pueden consultar en la la página https://developer.mozilla.org/es/docs/Web/HTTP/Status

Como se puede ver **nos devuelve un objeto `Response`** con **métodos para ver las cabeceras y el estado de la respuesta**. En este primer paso **no se retorna el resultado de la consulta**. Para ello, deberemos **emplear el método `json()`** que, a su vez, **devuelve otra promesa**:La salida será una promise cuyo resultado ya serái el json correspondiente

```
let f = fetch("https://dog.ceo/api/breeds/list/all");
f.then(
    result => console.log(result.json()),
    error => console.log("Error : " + error)
);
```

La salida:

```
> Promise {<pending>}
```

Por lo tanto podríamos modificar el código para leer el resultado de la siguiente forma;

```
fetch("https://dog.ceo/api/breeds/list/all").then(
    result => {
        result.json().then(
            result => console.log(result),
            error => console.log(error)
        )
    },
    error => console.log("Error : " + error)
);
```

NOTA : En este caso ya devuelve el resultado de la promesa de la transformación a JSON en result.

Dentro del atributo message se devuelve las razas como clave y las subrazas como valor en un array de strings.

La salida:

```
>{message: {…}, status: "success"}
message: {affenpinscher: Array(0), african: Array(0), airedale: Array(0), akita: Array(0), appenzeller: Array(0), …}
status: "success"
__proto__: Object
```

La información, en este caso, está en `message`, que es donde estarían las diferentes razas de perro, así que el código final:

```
fetch("https://dog.ceo/api/breeds/list/all").then(
    result => {
        result.json().then(
            result => console.log(result.message),
            error => console.log(error)
        )
    },
    error => console.log("Error : " + error)
);
```

La salida

```
>{affenpinscher: Array(0), african: Array(0), airedale: Array(0), akita: Array(0), appenzeller: Array(0), …}
affenpinscher: []
african: []
…
```

El problema con este tipo de soluciones es que cada vez que, dentro de una promesa, llamamos a otra, el código se complica y se hace más difícil de mantener.

Como ya sabemos, los métodos de las promesas retornan promesas cuyo valor es el objeto de retorno, así que la solución pasa por retornar las promesas (en nuestro caso el método `json()`) para poder encadenarlas con otro `then`: (ya que cada vez que veamos un then indica que la instrucción anterior está devolviendo una promesa).

EL fetch nos está recuperando la respuesta existosamente e intentamos convertir la respuesta a json(en esta transformación si se evalúa el caso exitoso y el erróneo)

```
fetch("https://dog.ceo/api/breeds/list/all")
.then( result => result.json() )
.then( result => console.log(result.message) )
.catch(error => console.log("Error : " + error));
```

La salida, evidentemente, es la misma. Como podemos comprobar vamos encadenando promesas (con el then) y puede llegar a complicarse mucho el código. Por eso es más fácil utilizar el Async/await, ofreciendo de esta forma un código más legible

```
async function recuperarRazas() {
      const resp = await fetch("https://dog.ceo/api/breeds/list/all");
      const obj = await resp.json();
      return obj;
}
```

Recordad que las funciones Async devuelven una promesa, cuyo resultado será obj. Posteriormente hay que gestionar la promesa: (usando await o then, recordar que el await si se usa dentro de una funcion, esta debe ser async)

Async / await con fetch

Aunque en los puntos anteriores hemos estado viendo promesas, obteniendo su resultado con .then, .catch y funciones callback. Es mucho más sencillo y utilizable usar Async/await.

Uno de los problemas con los que nos podemos encontrar a la hora de obtener información de Internet deriva de la naturaleza asíncrona de la operación que hace que no sepamos cuando vamos a obtener los resultados y, por tanto, no sepamos cuándo vamos a obtener la información.

Supongamos que queremos obtener las imágenes de la primera raza de perros. Primero obtendríamos todas las razas y luego cogeríamos la de la primera. El código sería algo así;

Un primer intento podría ser el siguiente;

```
let raza;
fetch("https://dog.ceo/api/breeds/list/all")
    .then( result => result.json() )
    .then ( r => raza = Object.keys(r.message)[0]);
fetch(`https://dog.ceo/api/breed/${raza}/images/random/10` )
    .then( result => result.json() )
    .then ( img => console.log(img));
```

Evidentemente esto **no funciona** dado que el **segundo `fetch` se ejecuta cuando el primero aún no ha finalizado** (y por tanto no ha actualizado `raza`).

Una solución sería **meter el segundo `fetch`** dentro del primero:

```
fetch("https://dog.ceo/api/breeds/list/all")
.then( result => result.json() )
.then ( r => {
    raza = Object.keys(r.message)[0];
    fetch(`https://dog.ceo/api/breed/${raza}/images/random/10` )
        .then( result => result.json() )
        .then ( img => console.log(img));
    }
)
```

La salida

```
>{message: Array(10), status: "success"}
```

El problema es que **el código se complica**, ¿Y si quisiéramos hacer algo con la primera foto? En el momento en que queremos ejecutar varias peticiones web que dependen unas de otras el código puede hacerse inmanejable.

La mejor solución, en estos caso, pasa por emplear `async/await`:

```
async function getImagenesRaza() {
    const responseRazas = await fetch("https://dog.ceo/api/breeds/list/all")
    const razasJson = await responseRazas.json();
    let raza = Object.keys(razasJson.message)[0];      //Nos quedamos con la primera raza
    const responseImagenes = await fetch(`https://dog.ceo/api/breed/${raza}/images/random/10`)
    //Coge las imágenes de la primera raza
    const responseImagenesJson = await responseImagenes.json();

    return responseImagenesJson.message;
}
getImagenesRaza().then(imagenes => console.log(imagenes));
// Normalmente, cuando usamos async / await de una manera profesional, se suele utilizar en dos partes,
 1- un async / await ( para recuperar la informacion por ejemplo y que nos devuelva  una promesa con el resultado que queramos).
2.- Al llamar a nuestra funcion (en nuestro ejemplo getImagenesRaza), coger el datos con .then y pintarlo o hacerlo que necesitemos.
Es decir, que cada funcion tenga una única responsabilidad. En este caso, tener una función para recuperar datos y otro para pintar los datos.
De tal manera que la función de recuperar datos sea más reutilizable
```

La salida:

```
(10)["https://images.dog.ceo/breeds/affenpinscher/n02110627_11365.jpg",
"https://images.dog.ceo/breeds/affenpinscher/n02110627_11853.jpg", ...]
```

NOTA: Es importante recalcar que `await` no se puede usar dentro de una función si ésta no está definida como `async`.